Cognitive Load from EEG + fNIRS — Complete Project Explanation

Everything this project is: the dataset, the pipeline from raw brain recordings to a served prediction, every model tried, what each file does, and the history of how it got here.

Companion documents: `API_AND_ARCHITECTURE.md` covers the HTTP API endpoint by endpoint. `BINARY_MODEL_IMPROVEMENT.md` covers the `research/` experiment framework, its leakage audit and its development/lockbox protocol. This one covers the whole project.

> **Correction (August 2026).** Earlier revisions of this file stated that the extracted feature data was absent and that the 3-class research checkpoint could not be regenerated. **The data is present**, under `ZDATASET/`, and is referenced from `data/` by symlink. Three windowed feature sets and the full-trial set are all available, so the pipeline is reproducible end to end. The sections below that discuss the missing `data/` are kept for history but no longer describe the current state of the repository; see §11.

---

1. What the project does

It estimates **mental workload** from two brain signals recorded at the same time, and it serves that model through a web app.

- **EEG** — electrical activity, fast (200 Hz), noisy, measures cortical rhythms.
- **fNIRS** — hemodynamic activity, slow (10 Hz), measures blood oxygenation (HbO = oxygenated, HbR = deoxygenated haemoglobin).

The two are complementary: EEG catches fast electrical shifts, fNIRS catches the slower metabolic response. Fusing them is the modelling premise.

The headline result is methodological, not a leaderboard score. Performance measured on *unseen people* (leave-one-subject-out) is far below what a conventional fixed train/test split suggests, because a fixed split lets the same person appear on both sides and the model exploits that person's individual physiological baseline. The project reports both numbers together, always.

---

2. The dataset

TU Berlin open dataset (Shin et al., 2018) — simultaneous EEG + NIRS recordings during an n-back working-memory task.

Property	Value
Subjects	26
EEG	30 channels @ 200 Hz
fNIRS	36 channels @ 10 Hz (HbO and HbR each)
Trial length	60 s task + 5 s baseline
Task	n-back working memory
Classes	0-back = **Low**, 2-back = **Medium**, 3-back = **High**

The n-back task is the workload manipulation: the participant sees a stream of items and must report whether the current one matches the item *n* steps back. Higher *n* = more items held in working memory = higher cognitive load. So the label is not a subjective rating — it is the experimentally imposed difficulty, which is what makes it usable as ground truth.

`Dataset description_MATLAB.pdf` in the repo root is the original documentation. Raw `.mat` files live under `data/` (gitignored, large, not distributed here).

Two feature sets exist in this project

A. Full-trial (the original 3-class pipeline) — one row per trial. 654 samples, 26 subjects, 180 EEG + 468 HbO features.

B. Sliding-window (the current pipeline) — each trial is sliced into overlapping windows, producing far more rows. **540 EEG + 936 fNIRS = 1476 features**, which is what the served models take:

- **540 EEG** = 30 channels × 18 features — 6 bands (delta, theta, alpha, beta, gamma, plus an engagement index) × 3 statistics (mean, std, slope).
- **936 fNIRS** = 36 channels × 26 features — 13 statistics each for HbO and HbR (mean, std, slope, min, max, ptp, auc, power, skew, kurtosis, early, late, delta). "early" and "late" capture the *shape* of the hemodynamic response over the window, which turned out to matter more than any EEG band.

Every column name is fixed and meaningful — `eeg_ch00_alpha_std` is channel 0's alpha-band variability. The model's input layer is bound to that exact ordering, which is why the upload endpoint validates columns strictly.

---

3. The pipeline, end to end

```
raw .mat files (EEG_01–26, NIRS_01–26)
        │
        │  preprocess_tuberlin.py — bandpass filter, epoch on NIRS markers so
        │  both modalities stay aligned, compute band powers + haemodynamic stats
        ▼
data/tuberlin_eeg_fnirs.csv          (full-trial features, 654 rows)
        │
        │  sliding_window_extractor.py / _v2.py — slice trials into overlapping
        │  windows, keep subject_id + trial_id so splits can group by them
        ▼
data/tuberlin_sliding.csv            (windowed features, 1476 cols)
        │
        │  data_loader.py — NaN handling → per-subject z-score → log compression
        │  → train-only fitted scalers.  Splits GROUPED BY SUBJECT.
        ▼
trainer.py + models/architectures.py — train the hybrid network
        │
        │  part1_rigor.py — 26-fold LOSO, significance tests, RF ensembling
        │  dann_ablation.py, hbr_ablation.py, classical_sweep.py, protocol_benchmark.py
        ▼
outputs/*.pt, outputs/*.json         (checkpoints + evaluation artifacts, gitignored)
        │
        │  export_for_serving.py / export_3class.py — freeze the chosen model
        ▼
serving/ and serving_3class/         (portable bundles, COMMITTED)
        │
        │  FastAPI loads each bundle once at startup
        ▼
backend/ (:8000)  ──HTTP──►  frontend/ (:5173)
```

Why the preprocessing order matters

`nan_to_num → per-subject z-score → log → saved scalers`

- **Per-subject z-score before log** was the single most important choice. It removes each person's individual baseline. It *lowers* fixed-split scores (0.4678 → 0.4064 macro F1) and *raises* LOSO scores (0.2982 → 0.3642) — i.e. it deletes a shortcut that never generalised. Doing it *after* the log was tested and rejected (0.3476 vs 0.3642).
- **Log compression** — EEG band powers are heavily right-skewed; `log1p` with non-negative clipping fixes that. fNIRS uses a *signed* log because HbO/HbR values go negative.
- **Scalers fitted on training folds only** — fitting on all data would leak test statistics into training.

---

4. The models

4.1 The hybrid network (the served model)

Defined in `models/architectures.py`, mirrored standalone in `backend/model_service.py` so serving never imports the training code.

```
EEG (540) ──► ModalityMLP ──┐
                            ├──► AttentionFusion ──► FC head ──► softmax
fNIRS (936) ► ModalityMLP ──┘
```

- **ModalityMLP** — input projection → *N* residual MLP blocks → output projection. One per modality, so each learns its own representation before they meet.

- **ResMLPBlock** — `Linear → BatchNorm → ReLU → Dropout → Linear → BatchNorm`, added back to its input. Residual connections keep gradients healthy in a small-data regime.
- **AttentionFusion** — the interesting part. It computes a **gate** per modality from the concatenation of both, then scales each modality by its own gate before fusing. The model learns *how much to trust EEG versus fNIRS*, and those gate values are exposed in every API response (`attention_weights`) — that's what the UI's modality bar shows.

> **Naming caveat:** the class is called `HybridCNNLSTM` and the config has `CNN_FILTERS` / `LSTM_UNITS`, but there is **no convolution and no LSTM**. The inputs are pre-extracted feature vectors, not sequences. The names are legacy from an earlier design; `CNN_FILTERS[-1]` is now just the projection width and `len(LSTM_UNITS)` is the residual block count. Worth knowing before reading the config.

Served hyperparameters differ between bundles:

	binary	3-class
`proj_dim`	96	16
`enc_dim`	48	48
`fc_units`	[48, 24]	[16]
dropout	0.5 / 0.6	0.5 / 0.6

4.2 The Random Forest half of the "ensemble"

The research pipeline averaged the hybrid network's softmax with a 400-tree Random Forest (seed 42) fitted on the same scaled features. `serving/rf_model.pkl` (46 MB, Git LFS) is that forest.

It is disabled in the live app. `backend/model_service.py` sets `self.rf = None` — "the new Hybrid model is much better alone." The config and several API responses still say `hybrid_rf_ensemble`, which is now a misnomer.

4.3 DANN — the novel contribution

`dann.py`, `dann_ablation.py`, written up in `NOVELTY_DANN.md`.

The project's ceiling is subject variability. **Domain-Adversarial Neural Networks** attack it head-on: a second head tries to predict *which subject* a sample came from, and a **gradient-reversal layer** sits between it and the encoder. Forward pass: identity. Backward pass: multiply the gradient by `-lambda`. The discriminator gets better at identifying subjects while the same signal pushes the encoder to *erase* subject identity. The result is a representation that keeps load information and discards person information (Ganin & Lempitsky, 2015).

`dann_ablation.py` runs the decisive 26-fold LOSO comparison against the baseline, plus a lambda sweep and a subject-invariance probe.

4.4 Classical baselines (all ruled out)

`classical_sweep.py` ran Random Forest, XGBoost and SVM under the same 26-fold LOSO protocol. Nothing beat the hybrid. `train_baseline.py`, `train_best.py`, `train_best_v2.py` and `train_with_pca.py` are earlier classical/PCA attempts, kept as the record of what was tried.

---

5. Results

The 3-class research model

Evaluation	Accuracy	Macro F1	AUC	Per-class recall [L, M, H]
Fixed split	0.4267	0.4064	0.6305	[0.563, 0.519, 0.188]
26-fold LOSO	0.3982 ± 0.074	0.3642 ± 0.089	0.6011 ± 0.080	[0.592, 0.325, 0.285]

Chance is 33%. So the model is real but modest, and **High load is hardest to detect** — recall around 19–29%.

The binary model (this branch)

Reframing to Low vs High (0-back vs 3-back) and adding sliding-window augmentation lifts LOSO performance substantially:

Config	Macro F1	Accuracy	AUC
Hybrid	0.650 ± 0.05	0.651	0.68
Hybrid + RF ensemble	0.650 ± 0.04	0.651	—

Chance is 50%. This is the honest cross-subject number for an easier task — not a fix for the 3-class problem, a different problem.

What was tried and rejected

Focal loss (0.3457), cross-attention fusion (0.3256), per-modality PCA (0.3228), `after_log` normalization stage (0.3476), manual class weights `[1.0, 1.1, 1.3]` (0.3329), and several window/stride variants — all below the 0.3642 reference. `RESULTS.md` records every one with its number, which is the point: the negative results are evidence, not clutter.

Explainability

- **SHAP** (`explain_model.py` , via `GradientExplainer` — `DeepExplainer` failed its additivity check) found fNIRS HbO response-*shape* features dominant: skew, early-window response, slope, max, mean, delta, kurtosis, AUC. EEG bands contributed much less.
- **Attention gates** (`visualize_attention.py`) agreed: mean fNIRS gate 0.514 vs EEG 0.478. Two independent methods pointing the same way is the reassuring part.

Why the literature reports 60–90%

`protocol_benchmark.py` answers this directly. Most published numbers use an easier protocol: binary instead of 3-class, and/or *within-subject* validation where the same person appears in train and test. Under matched protocols this project's numbers are comparable. The taxonomy is in `RESULTS.md` .

---

6. File-by-file guide

Data preparation

File	What it does
`preprocess_tuberlin.py`	Raw `.mat` → features. Filters, epochs EEG on **NIRS markers** so both modalities stay aligned with matching trial counts, computes band powers + haemodynamic stats.
`sliding_window_extractor.py`	Slices trials into overlapping windows, keeping `subject_id` / `trial_id` so splits can group by them.
`sliding_window_extractor_v2.py`	Variant that makes EEG a real temporal sequence (`eeg_t00_ch00_delta` , …) with per-subject baseline normalization against that subject's 0-back windows.
`data_loader.py`	The preprocessing contract: NaN handling, per-subject z-score, log transforms, train-only scaler fitting, subject-grouped splits. The upload service is verified byte-exact against this.
`extract_features.py`	Standalone feature extraction helpers.
`check_eeg.py` , `debug_eeg.py` , `diagnose_data.py` , `stats_check.py` , `eda.py`	Exploratory/diagnostic scripts written while chasing the EEG skew and label-alignment problems.

Configuration

File	What it does
`config.py`	The live config: paths, split sizes, architecture dims, optimizer, class weights, loss.
`config_improved.py` , `config_backup.py`	Snapshots kept so a baseline can be restored during debugging.

Training and evaluation

File	What it does
`models/architectures.py`	Model definitions (`build_model` , the hybrid, fusion variants).
`trainer.py`	Training engine — epoch loop, class weighting, early stopping, checkpoint selection by **validation macro F1**, progress bars.
`main.py`	Entry point that wires config → loader → model → trainer.
`evaluate.py`	Evaluation/report generation for a trained checkpoint.
`part1_rigor.py`	The rigor pass: 26-fold LOSO for three configs, paired significance tests, RF ensembling, appends the "Locked Model" section to `RESULTS.md` .
`phase3_experiments.py`	The tuning campaign — cheap wins, cross-attention, window/stride ablation, LOSO.
`dann.py` / `dann_ablation.py`	Domain-adversarial training and its 26-fold LOSO ablation + lambda sweep + invariance probe.
`hbr_ablation.py`	Does adding HbR back help? (It does: hybrid macro F1 +0.087, p=0.002.)
`classical_sweep.py`	RF / XGBoost / SVM under the same LOSO protocol. Ruled out.
`protocol_benchmark.py`	Protocol taxonomy — within-subject vs LOSO, binary vs 3-class.
`train_baseline.py` , `train_best.py` , `train_best_v2.py` , `train_with_pca.py`	Earlier classical/PCA pipelines, kept as record.
`validate_eeg_v2_gate.py` , `diagnose_eeg_v2_gate.py`	The EEG v2 feature gate (RF macro OVR AUC ≥ 0.55) and its diagnostics.
`explain_model.py`	SHAP explanations.
`visualize_attention.py`	Attention-gate visualization.
`test_model.py`	Load a `.pt` and run inference on the test set or any CSV.

Export and packaging

File	What it does
`export_for_serving.py`	Freezes the binary model into `serving/` — weights, config, scaler, label map, demo samples, upload schema, frontend artifacts.
`export_3class.py`	Same for `serving_3class/`.
`export_diagnostics.py`	Calibration/reliability diagnostics over the committed demo windows → `frontend_data/diagnostics.json`.
`export_research_data.py`	Consolidates DANN / HbR / protocol / classical artifacts → `frontend_data/research.json` for the Research page.
`prototype_936_serving.py`	Prototype of the 936-feature (EEG+HbO+HbR) served ensemble.
`compare_936_vs_live.py`	Compares that prototype against the live model *through the real backend service*, proving the backend can serve it.
`train_3class_from_demo.py`	**Added in this session.** Rebuilds a servable 3-class model from `demo_samples.json` — see §8.
`generate_synthetic_test_csvs.py`, `generate_synthetic.py`, `generate_dummy_data.py`	Synthetic fixtures for exercising the upload path. Explicitly **not** evidence of performance.

Backend (`backend/`, FastAPI)

File	Lines	What it does
`main.py`	437	The app: all routes, CORS, and per-bundle startup loading. Each bundle loads independently — one broken bundle can't take the API down.
`model_service.py`	227	Rebuilds the network from `model_config.json` + `model_weights.pt` **without importing the training pipeline**, runs inference, computes uncertainty (confidence, margin, entropy, abstain flags) and attention gates.
`data_service.py`	114	Reads `frontend_data/*.json` and `demo_samples.json`; serves metrics, confusion matrices, SHAP, attention, subject/session lists.
`upload_service.py`	130	CSV column validation and preprocessing — a self-contained reimplementation of `data_loader.py`, verified byte-exact (max abs diff 0.0).
`mat_extractor.py`	185	Raw `.mat` → features for the `.mat` upload endpoint.
`schemas.py`	55	Pydantic request/response models.

Frontend (`frontend/src`, React + Vite)

Pages — `Overview` (landing + model comparison), `Research` (490-line long-form dossier), `LivePrediction`, `SessionTrend`, `UploadData`, `ModelInsights`, `Methodology`.

Infrastructure — `App.jsx` (nav, routes, footer), `api/client.js` (every API call), `useApi.jsx` (fetch hook + loading/error components), `constants.js` (class colours, formatters).

`lib/` — `theme.jsx` (light/dark/system), `modelVersion.jsx` (**the 2-class ⇄ 3-class selection context**), `toast.jsx`, `chartTheme.js`, `exportData.js` / `exportImage.js` (CSV/PNG export).

Key components — `ModelSwitch` (the classification-mode control), `ModelProvenanceNote` (stand-in-model banner), `ConfusionMatrixChart`, `ProbabilityBars`, `TrendChart`, `ModalityContribution` (EEG vs fNIRS

gates), `UncertaintyPanel` + `ConfidenceGauge`, `ArchitectureDiagram`, `DannDiagram`, `CommandPalette` (⌘K), plus ~20 chart components under `components/research/` for the dossier.

---

7. How a prediction actually flows

```
User picks subject + window on /live, clicks Predict
   │
   ▼  POST /api/predict?version=binary   {"subject_id": 1, "window_index": 0}
backend/main.py  ──► get_services("binary")   picks the loaded bundle
   │
   ▼  data_service.get_window()   pulls the 1476 preprocessed floats
   ▼  model_service.predict_single()
        ├─ split into EEG[0:540] and fNIRS[540:1476]
        ├─ ModalityMLP each  →  AttentionFusion  →  FC head  →  softmax
        ├─ read the fusion gate values (modality attention)
        └─ compute confidence / margin / entropy / abstain
   │
   ▼  JSON: predicted_label, probabilities, attention_weights, uncertainty, truth
frontend renders ProbabilityBars + ConfidenceGauge + ModalityContribution
```

Nothing is hardcoded in the UI. Every number traces to a committed artifact or a live forward pass.

---

8. Project history — how it got here

The research phase (before this session)

1. Preprocessing built with EEG epoched on NIRS markers to guarantee alignment.
2. Early instability traced to skewed EEG band powers → log + RobustScaler.
3. Checkpoint selection switched from validation loss to validation macro F1 after models "looked fine" while collapsing a class.
4. Class-weight tuning tried and largely rejected; balanced train-fold weights kept.
5. **The fixed-split vs LOSO gap identified** — the project's central finding.
6. Per-subject normalization adopted (`before_log`), trading fixed-split score for genuine cross-subject gain.
7. Rigor pass: 26-fold LOSO, paired significance tests, RF ensembling.
8. Ablations: HbR (helps), DANN (the novel contribution), classical sweep (ruled out), protocol taxonomy (explains the literature gap).
9. Model frozen and exported to `serving/` ; FastAPI backend and React app built.
10. **The `binary-class` branch** reframed the task as Low vs High with sliding-window augmentation, lifting LOSO macro F1 to ~0.65, and archived the 3-class bundle as `serving_3class/` .

This session

1. **Environment set up** — `.venv` (Python 3.14, torch 2.13), npm packages, `git-lfs` installed and `git lfs pull` run. Three files were 133-byte LFS pointer stubs and would have crashed the backend. `scikit-learn` pinned to **1.8.0** because 1.9.0 warned about unpickling the fitted scalers.
2. **Startup crash fixed** — the API loaded both bundles eagerly, so the missing 3-class weights took the whole server down. Bundle loading is now per-bundle and non-fatal; an unavailable bundle returns 503 naming the missing file.

3. **Pages made resilient** — `Overview` and `LivePrediction` collapsed both bundles' errors into one flag and blanked entirely. Now only the *selected* model failing blanks a page.
4. **The model switch built** — new `GET /api/bundles`, `lib/modelVersion.jsx` context (persisted, resolved against what's actually servable), `ModelSwitch` component in the hero and nav, and every page rewired to follow the selection. Class names now come from the served model instead of a hardcoded `["Low","Medium","High"]` constant — the binary model had been rendering a phantom "Medium" bar.
5. **The 3-class model rebuilt** — see below.
6. **Two bugs found while verifying** — `UploadService` never stored `serving_dir`, so `GET /api/predict/upload/template` had been returning 500 on *both* versions (the "download example CSV" link was broken); and `/api/research/summary` 404s for 3-class, which now falls back to the binary copy since it's a project-level dossier.
7. **Documentation** — this file and `API_AND_ARCHITECTURE.md`.

The 3-class rebuild, and what it is

The original 3-class checkpoint (`outputs/hybrid_best.pt`) is **not in this repository on any branch**, and can't be regenerated because `data/` is absent. Without `model_weights.pt` the entire 3-class option was dead.

What *did* exist: `serving_3class/demo_samples.json` — 7650 real, labelled, fully preprocessed windows across 6 subjects. `train_3class_from_demo.py` trains the architecture from `model_config.json` on those, validating on subjects never trained on (18 and 24):

```
best epoch 10: held-out-subject acc 0.4133  macro F1 0.3875  (chance ≈ 0.333)
```

That lands in the same ~40–44% band as the original 3-class model — reassuring, but **it is a stand-in, not the research model**:

- trained on what were originally the fixed-split *test* windows, the only labelled 3-class data present;
- 6 subjects, not 26.

This is recorded in `model_config.json["provenance"]`, exposed by `/api/bundles`, and shown as a banner on Overview, Live Prediction and Model Insights — because those pages' charts still describe the *original* model. Drop the real checkpoint in and the banner disappears on its own.

The scaler was copied from `serving/` after asserting both bundles declare byte-identical feature columns. Scaling is label-agnostic, so this is sound.

---

11. The `research/` experiment framework (added August 2026)

Everything in this section is additive: `serving/`, `serving_3class/`, `backend/model_service.py` and the legacy training scripts are untouched, so the existing 2-class and 3-class demo behaves exactly as before.

What it is

A leakage-audited framework for improving the **binary** model without letting the evaluation flatter it, plus a run archive so every result traces to the code, config and data that produced it. Full detail in `BINARY_MODEL_IMPROVEMENT.md`; measured results in `RESULTS_BINARY_IMPROVEMENT.md`.

Module	Responsibility
`research/runctx.py`	run directories, status, metadata, NaN-safe JSON, failure preservation
`research/datamod.py`	loading, preprocessing contract, LOSO fold generation, per-fold scalers
`research/leakage.py`	the leakage assertions; raises and aborts on any failure
`research/archs.py`	model zoo — MLP and 1D-CNN encoders, concat/attention fusion, DANN
`research/train.py`	early stopping on validation macro F1, LR schedules, modality dropout
`research/metrics.py`	every metric, computed from predictions only
`research/plots.py`	every figure, written into the run that produced it
`research/explain.py`	SHAP, t-SNE, subject-invariance probe
`research/experiment.py`	one spec across one fold set
`research/campaign.py`	the staged search and the development/lockbox split
`run_campaign.py`	CLI entry point

The protocol

The 26 subjects are split once into an **18-subject development pool** (all experiment selection) and an **8-subject lockbox** (evaluated once, at the end, for the chosen configuration only). The headline 26-fold LOSO is still reported, labelled as selection-informed, so it stays comparable to the rest of the project's numbers.

The data question, settled

The three windowed feature sets differ only in window geometry:

Key	File	Rows (binary)	Geometry
`sliding_10s_5s`	`tuberlin_sliding.csv`	4,576	10 s window, 5 s stride
`sliding_10s_1s`	`tuberlin_sliding_augmented.csv`	21,216	10 s window, 1 s stride
`sliding_8s_4s`	`tuberlin_sliding_8s_4s.csv`	5,824	8 s window, 4 s stride

`config.py` defaults to the 1 s-stride "augmented" file, whose adjacent windows overlap by 90%. That inflates the row count ~4.6x without adding independent information. Under subject-grouped LOSO it is not leakage, but it is a real overfitting risk, so window geometry is run as a controlled experiment (`S10`) rather than assumed.

New API surface and pages

`GET /api/runs...` serves the archive; `/evaluation` and `/runs` render it. See `API_AND_ARCHITECTURE.md` §7.

---

9. Known issues and open work

Stale artifacts in the binary bundle. `frontend_data/confusion_matrices.json` is *entirely* 3-class (`labels: [Low, Medium, High]` , all matrices 3×3) even under `?version=binary` . `metrics.json` is mixed: `loso.hybrid` and `loso.ensemble_hybrid_rf` are genuinely binary, while `fixed_split.*` , `loso.fnirs_only` and `loso.hybrid_non_normalized` are stale 3-class. The UI contains this — charts render each matrix's own `labels` , and mismatched recall rows are dropped rather than mislabelled — but the real fix is regenerating them from a binary run of `export_for_serving.py` , which needs `data/` .

Misleading upload error text. It says "540 EEG + 936 HbO"; it is actually 540 EEG + 468 HbO + 468 HbR. The schema also still lists `hbr_` under `optional_ignored_prefixes` while `fnirs_columns` requires those columns.

The "ensemble" is not an ensemble. The RF half is disabled in code while config and API responses still say `hybrid_rf_ensemble`.

Uploads only accept TU-Berlin-format extracted features. This is not removable gatekeeping: the network's input layer is 1476 slots each bound to a specific measurement, and the scalers are per-column. Another dataset needs either the `.mat` extraction path (same rig), a column mapping (equivalent measurements), or its own trained model.

Per-subject normalization is a demo-mode simplification. A genuinely new person has no precomputed statistics, so the demo reuses each known subject's.

26 subjects is small for cross-subject claims; the 3-class stand-in sees 6.

The architecture names lie. `HybridCNNLSTM`, `CNN_FILTERS`, `LSTM_UNITS` — no convolution, no recurrence.

---

10. Quick reference

```
# Run the app
cd backend && ../.venv/bin/uvicorn main:app --reload --port 8000
cd frontend && npm run dev                    # http://localhost:5173

# Fresh clone needs LFS or the backend won't start
brew install git-lfs && git lfs install --local && git lfs pull

# Research reproduction (needs data/)
.venv/bin/python part1_rigor.py               # LOSO + significance + ensembling
.venv/bin/python dann_ablation.py             # the novel contribution
.venv/bin/python protocol_benchmark.py        # why literature reports 60-90%
.venv/bin/python classical_sweep.py           # XGBoost/SVM vs RF
.venv/bin/python export_for_serving.py        # repackage serving/
.venv/bin/python export_research_data.py      # rebuild research.json

# Rebuild the 3-class bundle from demo samples (no data/ needed)
.venv/bin/python train_3class_from_demo.py

# Binary-model improvement campaign (research/ framework)
.venv/bin/python run_campaign.py --tag my-run     # full staged campaign
.venv/bin/python run_campaign.py --stages S6      # one stage only
.venv/bin/python run_campaign.py --quick --limit-subjects 7 --max-epochs 6   # smoke test
```

Document	Contents
`README.md`	Setup and repo structure
`RESULTS.md`	Full results + every rejected candidate, with numbers
`NOVELTY_DANN.md`	The domain-adversarial contribution
`literature_survey.md`	Related work
`explanation/API_AND_ARCHITECTURE.md`	Endpoint-by-endpoint API reference
`explanation/PROJECT_OVERVIEW.md`	This file
`explanation/PROJECT_RECORD.md`	**The complete record** — architecture, protocol, every experiment, final results, limitations
`explanation/BINARY_MODEL_IMPROVEMENT.md`	The `research/` framework, protocol and leakage audit
`RESULTS_BINARY_IMPROVEMENT.md`	Measured results of the binary-model campaign